- **认识 jQuery**
  - 介绍
    - [jQuery](http://jquery.com/download/) 是一个 `Javascript` 专注于 DOM 操作的类库，通过一系列的封装，不仅简化了 DOM 操作，还处理了不同浏览器之间的兼容，极大的提升了编码的效率，其口号是 Write Less Do More!
  - 下载
    - jQuery 从形式上来看只是一个独立的 `.js` 文件，可以在其[官网](https://jquery.com/)免费下载使用，在其官网上提供了多个不同的版本供开发者下载，一般推荐下载最新版本（目前为 v3.5.1），值得注意的是 jQuery 自 v2.0 开始不再兼容 IE6/7/8。
    - 通常下载 jQuery 时需要有 3 个文件，分别为：
      - jquery-版本号.js
      - jquery-版本号.min.js
      - jquery-版本号.min.map
    - 其中 jquery-版本号.js 与 jquery-版本号.min.js 内部分代码是一致的，区别仅仅是 jquery-版本号.min.js 中不包含注释、换行、缩进等，甚至变量名也被处理成单个字母形式，这样做的目的是压缩代码量，使文件体积变小。
    - jquery-版本号.min.map 是官方在压缩代码时自动生成的一个文件，该文件中记录了 jquery 压缩前后的对应关系，浏览器调试时能够快速定位到出错代码的位置。
  - 体验
    - 首先将下载好的 jQuery 文件使用 `script` 标签引入到网页标签中，如下代码所示：

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>jQuery - 体验</title>
</head>
<body>

  <!-- 引入 jquery -->
  <script src="./jquery/jquery-3.5.1.min.js"></script>
</body>
</html>
```

    - 由于 jquery-版本号.min.js 的文件体积较小，所以推荐使用。
    - 接下来基于 jQuery 实现一个简单的 DOM 操作，如下代码所示：

```html
<!DOCTYPE html>
```

```html
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>jQuery - 体验</title>
  <style>
    .box {
      width: 200px;
      height: 200px;
      background-color: pink;
    }
  </style>
</head>
<body>
  <!-- 改变盒子的位置 -->
  <div class="box"></div>

  <!-- 引入 jQuery -->
  <script src="./jquery/jquery-3.5.1.min.js"></script>
  <script>
    // 基于 jQuery 的 DOM 操作
    let x = 0;
    $('.box').click(function () {
      $(this).animate({marginLeft: x += 40});
    })
  </script>
</body>
</html>
```

    - 假如基于原生 DOM 实现上面的交互效果不得不编写大量的代码，然而基于 jQuery 的实现只用了很少的代码，这便是开发中推荐使用 jQuery 的最主要原因。

- **基础概念——实例**
  - jQuery 是基于 Javascript 构造函数的原型对象实现的，通过为原型对象添加属性或方法的方式实现对 DOM 操作的封装，以下代码是 jQuery 核心代码的简化版本：

```html
<script>
  // 定义了全局的构造函数 jQuery
  function jQuery(selector) {
    // new 关键字创建了一个实例
    // 此时 init 方法也被当成了构造函数使用
```

```
    return new jQuery.fn.init(selector);
  }

  // 定义别名
  jQuery.fn = jQuery.prototype;

  // 在 jQuery 的原型上添加了方法 init
  jQuery.fn.init = function (selector) {
    // 一些操作
  }

  // 建立关联
  jQuery.fn.init.prototype = jQuery.prototype;

  jQuery.prototype.css = function () {
    // 此处封装样式操作的逻辑
  }

  // 为 jQuery 定义别名
    let $ = jQuery;
</script>
```

- 总结：
  - 网页中引入 jQuery 后会得到一个全局的函数 `jQuery` 或 `$`
  - `$` 是 jQuery 函数的别名，使用 `$` 比使用 `jQuery` 更方便
  - 每次调用 `jQuery` 或 `$` 函数都会得到一个**新的实例**
  - 原型对象中存在许多的方法，调用这些方法实现 DOM 的各种操作

- **基础概念——选择器**
  - 说明：
    - jQuery 中的选择器是用来获取 DOM 节点的，其功能类似于原生的 `querySelectorAll` 方法，jQuery 中所支持的选择器与 CSS 的选择器几乎一致。
    - 选择器：jQuery通过元素的选择器获取元素、注意jQuery获取的是伪数组形式的，称为jQuery的对象
  - 标签选择器

```
<p>jQuery 封装了大量 DOM 操作的方法，极大的提升了开发效率！</p>
<p class="slogan">jQuery 的口号是 Write Less Do More!</p>
<p id="compat">jQuery 内部对浏览器之间的兼容也做了相关的处理。</p>

```

```
<script src="./jquery/jquery-3.5.1.min.js"></script>
<script>
  // 标签选择器
  $('p').css('color', 'red');
</script>
```

- 类选择器

```
<p>jQuery 封装了大量 DOM 操作的方法，极大的提升了开发效率！</p>
<p class="slogan">jQuery 的口号是 Write Less Do More!</p>
<p id="compat">jQuery 内部对浏览器之间的兼容也做了相关的处理。</p>

<script src="./jquery/jquery-3.5.1.min.js"></script>
<script>
  // 类选择器
  $('.slogan').css('color', 'blue');
</script>
```

- id 选择器

```
<p>jQuery 封装了大量 DOM 操作的方法，极大的提升了开发效率！</p>
<p class="slogan">jQuery 的口号是 Write Less Do More!</p>
<p id="compat">jQuery 内部对浏览器之间的兼容也做了相关的处理。</p>

<script src="./jquery/jquery-3.5.1.min.js"></script>
<script>
  // id 选择器
  $('#compat').css('color', 'green');
</script>
```

- 后代选择器

```
<p>jQuery 封装了大量 DOM 操作的方法，极大的提升了开发效率！</p>
<p class="slogan">jQuery 的口号是 Write Less Do More!</p>
<p id="compat">jQuery 内部对浏览器之间的兼容也做了相关的处理。</p>

<script src="./jquery/jquery-3.5.1.min.js"></script>
```

```
<script>
  // 后代选择器
  $('body p').css('fontSize', 20);
</script>
```

- 总结：
  - 将 CSS 的选择器以参数形式传给 jQuery 函数后便可以获取相应的 DOM 节点
  - 通过 jQuery 选择器获取 DOM 节点的同时也得到了一个 jQuery 实例
  - 调用实例方法完成相应的 DOM 操作
  - 注：几乎所有 CSS 选择器都可以被 jQuery 支持。

- **基础概念——对象**
  - 虽然在 jQuery 中利用选择器能获取到相应的 DOM 节点，但此时的 DOM 节点并非是原生的 DOM 对象，因此也就无法直接调用原生的 DOM 方法，如下代码所示：

```
<ul>
  <li>html</li>
  <li>css</li>
  <li>javascript</li>
</ul>
<!-- 引入 jQuery -->
<script src="./jquery/jquery-3.5.1.min.js"></script>
<script>
  // 这样写是错误的！！！
  $('li').style.color = 'red';
</script>
```

  - 在 jQuery 中利用选择器获取的 DOM 节点 jQuery 构造函数的实例对象，只能调用实例的方法才能完成 DOM 操作，正确的用法如下所示：

```
<ul>
  <li>html</li>
  <li>css</li>
  <li>javascript</li>
</ul>
<!-- 引入 jQuery -->
<script src="./jquery/jquery-3.5.1.min.js"></script>
<script>
```

```
  // 正确的用法
  $('li').css('color', 'red');
</script>
```

- 我们将 jQuery 获取的 DOM 节点对象称为 jQuery 对象，`css` 方法是 jQuery 对象的方法，而 `style` 是原生 DOM 对象中的一个属性。
- 在 jQuery 中并非只能通过选择器获取 jQuery 对象，它还可以将任意原生 DOM 对象转换成 jQuery 对象，如下代码所示：

```
<h4>将原生 DOM 对象转换成 jQuery 对象后，便可以调用其实例中的方法了。</h4>

<!-- 引入 jQuery -->
<script src="./jquery/jquery-3.5.1.min.js"></script>
<script>
  // 原生 DOM 对象
  let h4 = document.querySelector('h4');

  // 将原生 DOM 做为参数传入 $ 函数，自动将其转换为 jQuery 对象
  $(h4).css('color', 'red');
</script>
```

- 注：由于 jQuery 实例中包含了许多关于 DOM 操作的方法，因此基于 jQuery 开发时会将原生 DOM 对象转换成 jQuery 对象后再使用。

- **实例方法——事件**
  - 快捷方法
    - 在 jQuery 中以原生事件类型的名称为依据封装了相对应的事件处理方法，如下代码所示：

```
<div class="box"></div>

<script src="./jquery/jquery-3.5.1.min.js"></script>
<script>
  // 为 .box 添加单击事件
  $('.box').click(function () {
    console.log('.box 盒子被点击了...');
  });

  // 为 .box 添加鼠标移入事件
```

```
  $('.box').mouseover(function () {
    console.log('鼠标停留在 .box 盒子上了...');
  });
</script>
```

- 我们将以原生事件类型名称做为方法名为 DOM 添加事件的方式称为快捷方法。
- 总结：
  - 语法简洁，接收回调函数做为参数，事件触发时该回调函数被执行
  - 事件触发后回调函数中的 `this` 指向添加事件的原生 DOM
  - `$(this)` 是将原生 DOM 对象转换为 jQuery 对象

- 基础方法
  - 在 jQuery 中使用快捷方法能够快速为 DOM 节点添加事件监听，然而灵活性方面略显不足，因此 jQuery 又提供了较为灵活的 `on/off`、`bind/unbind`、`one` 方法处理 DOM 事件。

```
<div class="test"></div>
<button>付款</button>

<script src="./jquery/jquery-3.5.1.min.js"></script>
<script>
  // 使用基础方法 on 或 bind 添加事件监听
  $('.test').on('click', function () {
    console.log('.test 盒子被单击了...');
  })

  $('.test').bind('dblclick', function () {
    console.log('.test 盒子被双击了...');
  })

  // 使用基础方法 off 或 unbind 移除事件监听
  $('.box').off('click'); // 移除单击事件
  $('.box').unbind('mouseover'); // 移除鼠标悬停事件
  $('.test').off(); // 移除所有事件

  // 一次性事件
  $('button').one('click', function () {
    console.log('付款中...');
  })
</script>
```

        - 总结：
            - `on`、`bind`、`one` 语法结构与 `addEventListener` 类似，分别接受事件类型和回调函数做为参数
            - `on` 和 `bind` 含义基本一致推荐使用 `on` 添加事件监听，`one` 添加的事件监听只生效 1 次
            - `off` 和 `unbind` 用来移动 DOM 的事件，无论以何种方式添加的事件均可被移除，推荐使用 `off` 方法
            - 事件回调函数中的 `this` 仍然指向的是事添加事件监听的原生 DOM
    - 自定义事件
        - 在 jQuery 中除了支持原生 DOM 事件类型外，还允许开发者自定义事件类型，如下代码所示：

```
<script src="./jquery/jquery-3.5.1.min.js"></script>
<script>
  // 自定义事件类型 myevent
  $('.demo').on('myevent', function () {
    console.log('myevent 是自定义事件类型...');
  })

  // 自定义事件必须通过 trigger 才能被触发
  $('.demo').trigger('myevent');
</script>
```

        - 总结：
            - 自定义事件类型语法与原生事件类型一致
            - 自定义事件只能通过 `trigger` 方法才能被触发
            - `trigger` 方法也能用来触发原生事件类型
            - 注：自定义事件是一种高级的用法，不必深入理解只需要了解其基本用法即可。

- **实例方法——样式**
    - 行内样式
        - 通过 `css` 方法动态修改 DOM 的行内样式。

```
<div class="demo"></div>
<button class="btn1">改变上面盒子的样式</button>

<script src="./jquery/jquery-3.5.1.min.js"></script>
<script>
  // 单击按钮时改变 .demo 的样式
  $('.btn1').click(function () {
    // 调用 css 方法改变 .demo 的样式属性
```

```
    $('.demo').css({
      backgroundColor: 'blue',
      width: 240
    })
  })
</script>
```

- 总结：
  - 接受两个参数（分别为样式属性和值）时，逐一改变网页元素单个的样式属性
  - 接受一个参数时（对象类型），同时修改网页元素的多个样式属性
  - 样式属性值为长度时，省略长度单位 `px`
  - 接受单个样式属性做为参数时，能够获取该属性对应的值
  - 注：css 方法对应原生 DOM 操作中的 `style` 属性
- 类名操作
  - jQuery 中封装了为网页元素添加、移除、检测、切换类名的方法。

```
<div class="test"></div>
<button class="add">添加类名</button>
<button class="remove">移除类名</button>
<button class="has">检测类名</button>
<button class="toggle">切换类名</button>

<script src="./jquery/jquery-3.5.1.min.js"></script>
<script>
  $('.add').click(function () {
    // 添加 active 类名
    $('.test').addClass('active');
  })

  $('.remove').click(function () {
    // 移除 active 类名
    $('.test').removeClass('active');
  })

  $('.has').click(function () {
    // 检测是否包含 active 类名
    alert($('.test').hasClass('active'));
  })

```

```
  $('.toggle').click(function () {
    // 切换 active 类名
    $('.test').toggleClass('active');
  })
</script>
```

- 总结：
  - `addClass` 方法为网页元素添加一个或多个类名，多个类名用空格进行分隔
  - `removeClass` 方法移除网页元素的一个或多个类名，多个类名用空格进行分隔
  - `hasClass` 方法检测网页元素是否包含某个特定的类名
  - `toggleClass` 方法切换（添加/移除）一个或多个类名，多个类名用空格进行分隔
  - 注：以上 4 个方法的所实现的功能分别对应原生 DOM 中的 `classList` 的 `add`、`remove`、`contains` 和 `toggle` 4 个方法。

- **实例方法——链式**
  - 链式也叫链式操作，它是 jQuery 中提供的一种简便的语法结构，可以在一定程度上精简代码、增强代码的可读性，如下代码所示：

```
<style>
  .active {
    background-color: red;
  }
</style>

<div class="box"></div>
<button>点一下</button>

<script src="./jquery/jquery-3.5.1.min.js"></script>
<script>
  // 为按钮添加单击事件
  $('button').click(function () {
    // 连续调用多个实例方法
    $('.box')
      .css('width', 240) // 改变 style 属性
      .addClass('active') // 添加类名
      .click(function () { // 添加单击事件
        $(this).off('click');
        console.log('单击事件被触发...');
    })
  })
```

```
</script>
```

- 总结：
  - 链式操作只是一种简便的语法结构
  - 链式操作时实例方法自左向右依次被执行

- **实例方法——查找**
  - 父子关系
    - `find` 方法，参照某父元素查找其**后代元素**，该方法以选择器做为参数，在其后代元素查找符合选择器条件的网页元素，其用法如下代码所示：

```
<script>
  // 参照某父元素查找其【后代元素】
  $('.course').find('li').css('color', 'red');
</script>
```

    - `children` 方法，参照某父元素查找其**子元素**，其用法如下代码所示：

```
<script>
  // 参照某父元素查找其【子元素】
    $('.box').children().css('color', 'red');
</script>
```

      - 注：`children` 方法还允许传入选择器获取指定的子元素。
    - `parent` 方法，参照某个子元素查找其**父元素**，其用法如下代码所示：

```
<script>
  // 参照某个子元素查找其【父元素】
  $(this).parent().css('background-color', 'pink');
</script>
```

    - `parents` 方法，参照某个子元素查找其**祖先元素**，其用法如下代码所示：

```
<script>
  // 参照某个子元素查找其【祖先元素】
```

```
    $(this).parents().css('background-color', 'pink');
</script>
```

- 注: `parents` 方法还允许传入选择器获取指定的祖先元素。

- 兄弟关系
  - `siblings` 方法，查找所有同级结构关系的元素，其用法如下代码所示：

```
<nav>
  <a href="javascript:;">体育新闻</a>
  <a href="javascript:;">娱乐新闻</a>
  <a href="javascript:;">国际新闻</a>
  <a href="javascript:;">国内新闻</a>
</nav>
<script>
  $('nav a').click(function () {
    $(this)
      .addClass('active')
      .siblings() // 查找到同级的所有兄弟元素
      .removeClass('active');
  });
</script>
```

- 注: `siblings` 方法还允许传入选择器获取指定的兄弟元素。

  - `prev` 方法，查找当前元素之前的同级结构元素，其用法如下代码所示：

```
<script>
  // 最后一个 li 元素之前的元素
    $('li:last-child').prev().css('color', 'red');
</script>
```

  - `next` 方法，查找当前元素之后的同级结构元素，其用法如下代码所示：

```
<script>
  // 第一个 li 元素之后的元素
    $('li:first-child').next().css('color', 'red');
```

```
</script>
```

- **实例方法——筛选**
  - 大多数情况下 jQuery 获取的网页元素多以集合（伪数组）形式存在，同时 jQuery 还允许基于这个集合进一步筛选出更精确的网页元素。
  - `first` 方法，筛选出集合中的第一个元素

```
<ul>
  <li>html</li>
  <li>css</li>
  <li>javascript</li>
</ul>
<script>
  // 查找到第一个 li 元素
  $('li').first().css('color', 'red');
  $('ul').children().first();
</script>
```

  - `last` 方法，筛选出集合中的最后一个元素

```
<ul>
  <li>html</li>
  <li>css</li>
  <li>javascript</li>
</ul>
<script>
  // 查找到最后一个 li 元素
  $('li').last().css('color', 'blue');
  $('ul').children().last().css('color', 'blue');
</script>
```

  - `eq` 方法，以索引为依据筛选集合中的元素

```
<ul>
  <li>html</li>
  <li>css</li>
```

```
  <li>javascript</li>
</ul>
<script>
  // 以索引为依据筛选集合中的元素
  $('li').eq(1).css('color', 'green');
  $('ul').children().eq(1).css('color', 'green');
</script>
```

- **实例方法——属性**
  - jQuery 中封装了设置和读取网页元素属性的方法，其用法如下代码所示：

```
<img
  data-src="./images/card.gif"
  src="./images/placeholder.png"
  alt=""
>
<button>换图</button>
<script src="./jquery/jquery-3.5.1.min.js"></script>
<script>

  $('button').click(function () {
    // 读取 data-src 中图片路径
    let imgPath = $('img').attr('data-src');
    // 设置 src 属性
    $('img').prop('src', imgPath);
  })
</script>
```

  - 总结：
    - prop()操作固有属性、attr()操作自定义属性（固有属性也可以操作）、data()操作data-属性
    - 接受属性名为参数时用于获取该属性对应的值，接受属性和值两个参数时用于设置该属性且赋值
    - 注：针对自定义属性可以使用 `data` 方法。

- **实例方法——文本**
  - jQuery 中封装了设置和读取网页元素文本内容的方法，其用法如下代码所示：

```
<div class="box"></div>
```

```
<script>
  $('.box').html('<h4>学习jQuery</h4>');
  $('.box').text('<h4>学习jQuery</h4>');
</script>
```

- 总结：
  - `text` 相当于原生的 innerText ，它无法解析 html 标签
  - `html` 相当于原生的 innerHTML，能够解析文本中的 html 标签
  - `html` 和 `text` 方法不传入参数时用于读取文本内容

- **入口函数**
  - 描述：页面加载完成之后触发的函数
  - 第一种：

```
$(function () {
    $('div').css('background', 'red');
});
```

  - 第二种：

```
$(document).ready(function () {
    $('div').css('background', 'blue');
})
```